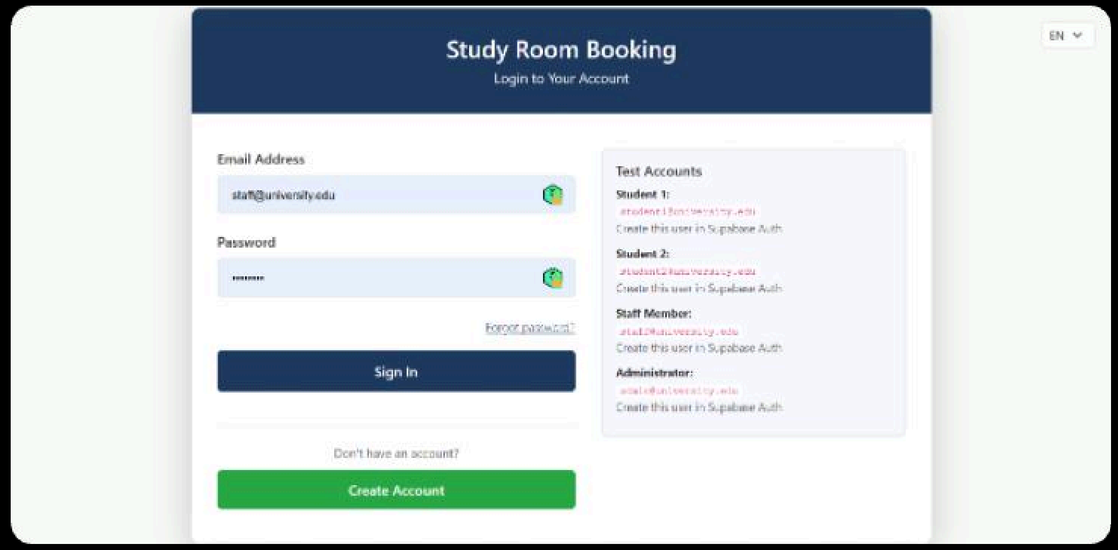
Study Room Booking & Management System

Team Members
Phuong Thi Mai Nguyen – student ID: 23197030
Benjamin Macpherson – student ID: 23208226
Thanh Tran– student ID: 24247169

Course Lead: Abubakar Siddique
Lecturer: Trung Nguyen
TA: Aarti Pande



Agenda:

1. **Project & requirements | Slides 3 to 5**
   Problem, requirements, prototype & what changed
2. **Testing & quality | Slides 6 to 8**
   Test approach, results, performance and security
3. **Automation & CI/CD | Slides 9 to 10**
   The pipeline, quality gates and evidence
4. **Defects & release decision | Slides 11 to 12**
   Root causes, corrective actions and readiness
5. **AI, future work & team| Slides 13 to 15**
   Copilot use, improvements, contributions and lessons

Part I | Project & requirements

The Problem: Manual room booking does not scale

Manual booking causes double bookings, extra admin work and unclear room availability.

Faster delivery does not guarantee quality, so the project applies QA practices to a real booking system.

Stakeholders

Students search, book, modify and cancel rooms.

Academic staff create recurring bookings.

Administrators manage rooms, access rules and overrides.

In Scope

1. Search, booking
2. History
3. Email confirmation
4. Recurring bookings
5. Admin functions
6. English & Vietnamese

Out of Scope

1. Utilisation reports (FR12)
2. Full security assessment
3. Keyboard navigation testing
4. Deployment to production

The solution

One web platform to search, book, modify and cancel rooms, with admin tools, a database and QA checks built in.

Part I | Project & requirements

11 of 12 functional requirements delivered

ID	Quality	Complete
FR1	Students can search available rooms using date, time, location, capacity and room type filters	Completed
FR2	Students can book an available room	Completed
FR3	The system shall validate the availability of study rooms before confirming a booking	Completed
FR4	The system will validate programme or major eligibility before allowing specialised room bookings	Completed
FR5	Student will receive booking confirmation via email	Completed
FR6	Student can modify future bookings	Completed
FR7	Student can cancel future bookings	Completed

ID	Quality	Complete
FR8	Student can view booking history	Completed
FR9	Academic staff can create recurring bookings	Completed
FR10	Administrators can manage room information and room access rules	Completed
FR11	Administrators can override or cancel bookings when necessary	Completed
FR12	Administrators can generate room utilisation reports	Deferred

Part I | Project & requirements

07 non-functional requirements delivered

Summary of Functional (FR1 to FR12)

Students: search, book, validate availability and eligibility, email confirmation, modify, cancel, history.

Staff: recurring bookings.

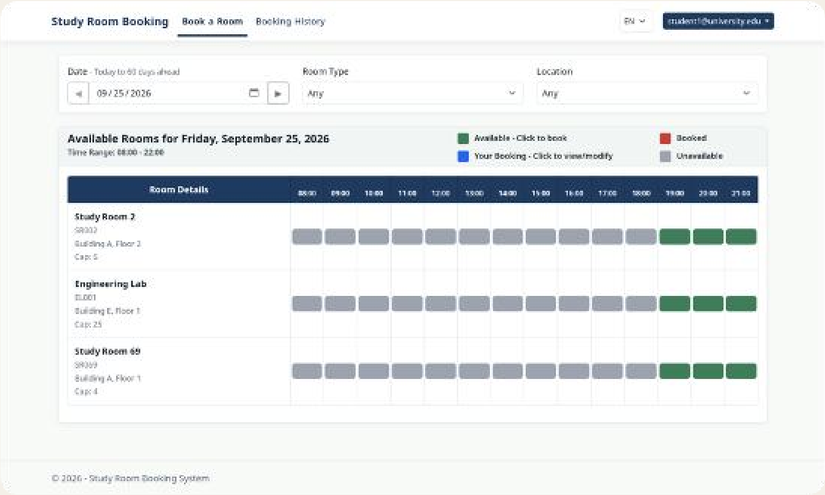
Admins: manage rooms and access rules, override bookings.

Deferred: FR12 utilisation reports.

ID	Quality	Measure
NFR1	Performance	Search ≤ 2 s, 50+ reservations
NFR2	Reliability	No duplicate bookings
NFR3	Security	Admin access for admins only
NFR4	Usability	First booking ≤ 4 min unassisted
NFR5	Maintainability	Modular architecture
NFR6	Accessibility	Keyboard use, EN/VI language switch
NFR7	Availability	Operational during demos

Part I | Project & requirements

The final prototype and what changed



Changes after Assessment 1

Database: in-memory to Supabase

Pages merged: Search with Bookings, My Bookings with History

Scope: FR12 utilisation reports deferred

Owner	Feature	What it does
Phuong	Database	Supabase and EF repositories
Phuong	Email confirmation	SMTP email with booking reference
Benjamin	Admin functions	Rooms, bookings, user roles
Benjamin	GUI improvements	Merged pages, design system
Thanh	Normal booking	Pop-up create and modify
Thanh	Recurring booking	Series that skips conflicts
Thanh	Languages	English and Vietnamese switch

Part II | Testing and quality

Automate the logic, check the rest by hand

20

test cases, TC-01 to TC-20

12 automated (60%) and 8 manual (40%), covering FR1 to FR11 and all seven NFRs.

- **Automated: MSTest and in-memory SQLite**

  Booking rules, access control, cancellation, history and a performance check. Each test gets a fresh, isolated database.

- **Manual: what needs eyes and judgement**

  Email delivery, recurring bookings, admin overrides, usability, Vietnamese wording and the database migration.

- **Both positive and negative cases**

  Double booking, past dates, missing student ID, restricted rooms and unauthorised cancellation.

Part II | Testing and quality

All 20 tests passed, but the suite is thin

20 / 20
test passed, 0 failed

94.7%
requirements with a passing test (18/19)

33.2%
line coverage, CI minimum is 30%

Key findings

- Regression tests plus TC-20 showed double-booking prevention survived the database migration.
- Manual testing found an ambiguous Vietnamese word that automation could not.

Remaining risks

- About one test per requirement.
- Tests run on SQLite, not the real database.
- Keyboard navigation and full security untested.

Part II | Testing and quality

Performance passed, security only partly tested

PERFORMANCE | NFR1

721 ms

Average search time with 98 reservations, against a 2,000 ms limit.

Evidence: three timed runs of 622, 760 and 781 ms in browser dev tools. Automated test TC-17 also passes.

Limit: measured on localhost, so no network delay or heavy load.

Next: measure again once deployed, with many users at once.

SECURITY | NFR3

9/9

Admin URLs blocked for students, staff and logged-out users. TC-15 also passes.

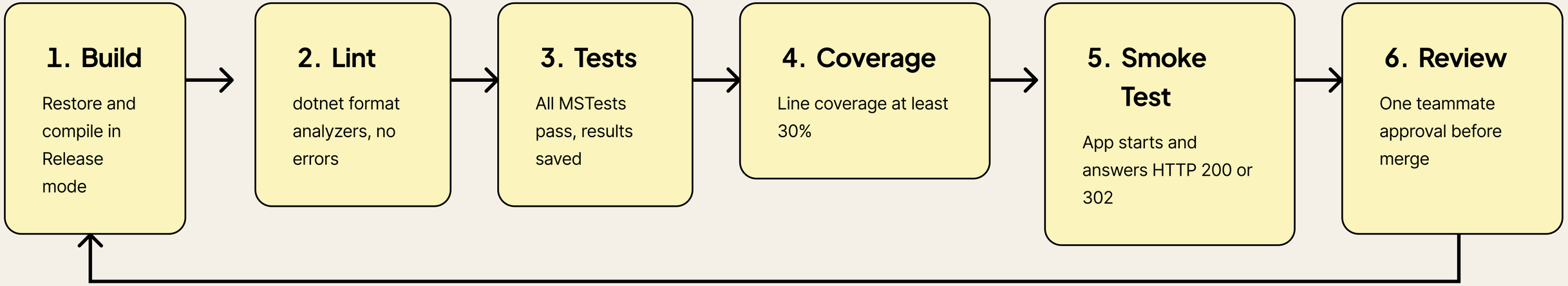
Evidence: Room Management, User Roles and Booking Management pages each redirected away.

Not tested: direct admin requests, session tampering, SQL injection, weak passwords.

Next: add these tests before any real release.

Part III | Automation and CI/CD

Six quality gates in the CI pipeline

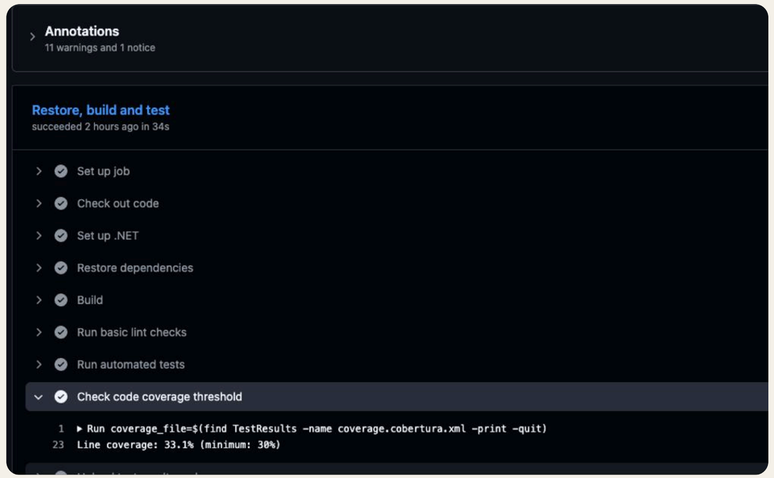


Regression tests plus TC-20 showed double-booking prevention survived the database migration. Manual testing found an ambiguous Vietnamese word that automation could not.

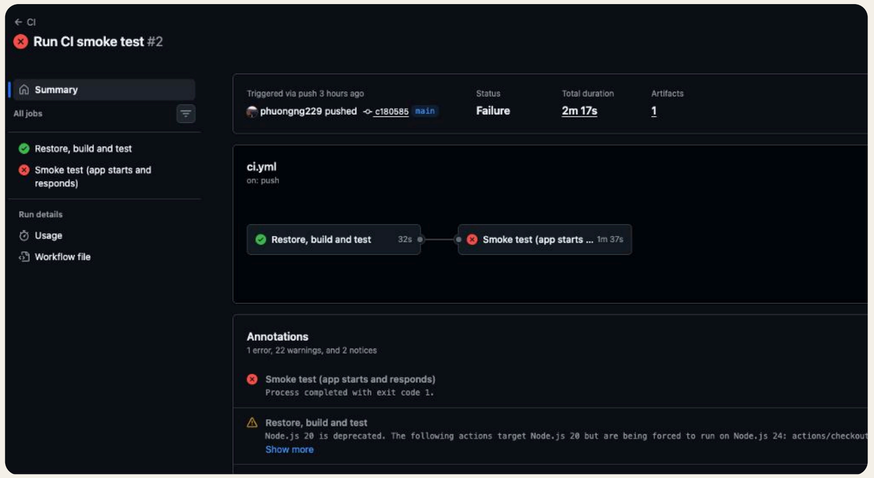
The database connection string lives in a repository secret, never in the code.

Part III | Automation and CI/CD

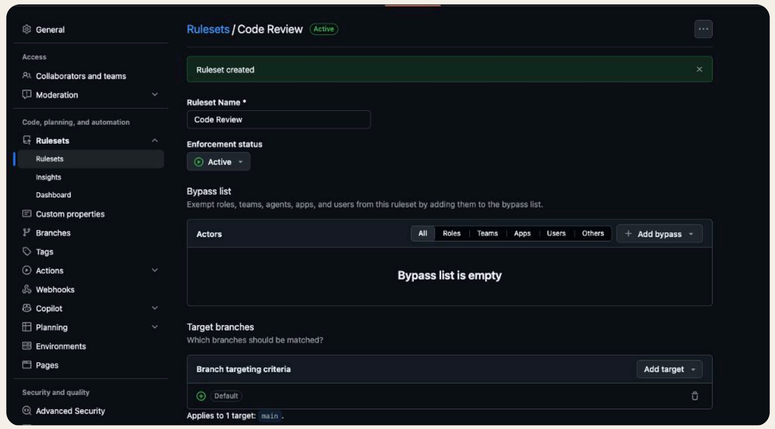
Evidence: the gates pass, fail and block



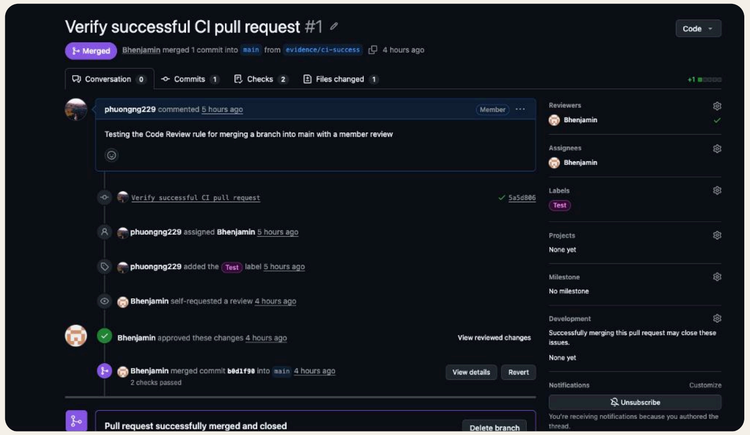
Passing run: every step green, coverage checked against 30%.



Failed run: the first smoke test failed because the connection string was missing.



Setup branch rule must have one reviewer approved before merging into main.



Code review: pull request approved by a teammate, then merged.

Part IV | Defects and release

Three defects found, root causes traced, all closed

ID	Defect	Severity	Root cause	Corrective action
DEF-01	Possible double bookings after database migration	High	New boundary between booking logic and storage	Added overlap tests, reran regression tests, TC-20
DEF-02	"Free" read as no cost in Vietnamese	Low	Literal translation, not checked in context	Changed wording, native speaker review
DEF-03	App failed to start though unit tests passed	High	Unit tests do not cover start-up or configuration	CI smoke test, secret passed correctly

0 CRITICAL

2 high | 1 low | All 3 closed

Lesson

Each defect was found by a different kind of check, so no single technique is enough.

Part IV | Defects and release

Not ready for release yet

20 / 20
test passed, 0 failed

94.7%
requirements with a passing test (18/19)

11 / 12
features done

33.2%
line coverage, CI minimum is 30%

Evidence for

- 11 of 12 features done
- 20 of 20 tests passed
- 3 defects found, all closed
- No critical defects
- Search 721 ms, limit 2 s

Evidence against

- FR12 reports not built
- About one test per requirement
- 33% coverage, on SQLite only
- Security: admin URLs only
- Keyboard use not tested

Before release

- Add the missing security tests
- Test on the real database
- Raise coverage above 30%
- Build FR12 reports
- Test keyboard navigation

Decision: Not ready for real students. Re-assess once the list above is done.

Part V | AI, future and team

GitHub Copilot: fast to start, never trusted unchecked

Used for

- Database and authentication
- Booking and recurring workflows
- Admin features and UI rework
- Test scenarios and CI workflow

What went wrong

- Admin code failed to build three times
- UI rework bypassed the login page
- Old test method broke on MSTest 4
- CI assumed no configuration needed

How we handled it

- Checked against requirements
- Ran tests and CI on every change
- Kept secrets out of code
- Peer review before merge

Lesson

AI output is only as good as the acceptance criteria and checks around it.

Part V | AI, future and team

Future improvements: toward continuous quality

Short Term | Tighten the gates

- Raise coverage to 60%, then 70%
- Full format check in lint
- Track defects as GitHub Issues
- Pull requests from sprint one
- One test per acceptance criterion

⇒ **Fixes:** 33% coverage and defects found late.

Medium Term | Test the real thing

- Run tests on the real database
- Concurrency tests for bookings
- Browser end-to-end tests
- Security tests and scanning
- Load test with many users

⇒ **Fixes:** tests ran on SQLite, security and load barely tested.

Longer Term | Close the Loop

- Staging deployment
- Logging and health checks
- Complete FR12 reports
- Accessibility and keyboard tests
- Monitor bookings after release

⇒ **Fixes:** no deployment, FR12 missing, keyboard untested.

Goal

Every change is checked automatically before it merges, and quality is monitored after release.

Part V | AI, future and team

Contributions and lessons learned

Phuong

Built: database, authentication, email confirmation, CI pipeline.

Lesson:

- Passing unit tests does not mean the app works: the smoke test failed first.
- Regression tests gave confidence after the database move.
- Check every AI suggestion, especially credentials.
- Start pull requests and review in sprint one.

Benjamin

Built: admin features, page layout, design system, test database, quality testing.

Lesson:

- Clear acceptance criteria give AI tools something to match.
- Quality metrics made the release decision stronger.
- Learn more security testing: our tests missed a lot.

Thanh

Built: normal and recurring booking, language switch, booking validation.

Lesson:

- Plan early: moving to popups later meant big rework.
- Test features together, not one by one.
- Switch approach sooner when one is failing.
- Review and test AI code before merging it.

As the team

Quality comes from several kinds of checking together (e.g. Unit Test, Regression Test, Smoke Test, Automated Test, Integration Test, and Manually Test). Start quality practices on day one. And we should be honest about what the tests do not cover.

THANK YOU

Thank you for your time and guidance throughout the course!